

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПАРАЛЕЛНА РАМКА ЗА МЕТАЕВРИСТИЧНА
ОПТИМИЗАЦИЯ НА КОМБИНАТОРНИ
ЗАДАЧИ: ПРОЕКТИРАНЕ, ПРОГРАМНА
РЕАЛИЗАЦИЯ И ЕКСПЕРИМЕНТАЛНА
ОЦЕНКА НА ПРОИЗВОДИТЕЛНОСТТА**

КУРСОВ ПРОЕКТ

по дисциплина „Метаевристика“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова, Доц. д-р Аделина Алексиева, Доц. д-р Георги Запрянов

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Комбинаторна оптимизация и мястото на метаевристиките	3
1.2. Метаевристики, включени в проекта	3
1.3. Паралелни схеми за изпълнение	4
1.4. Оценка на производителността	4
1.5. Еталонни набори от инстанции	5
1.6. Изводи от прегледа	5
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Език и среда за изпълнение	6
2.2. Модел на паралелизъм	6
2.3. Представяне на задачата пред алгоритъма	7
2.4. Стратегия за обмен между нишките	7
2.5. Обобщение на избора	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Слоеви и отговорности	8
3.2. Договор за задача	8
3.3. Договор за метаевристика	9
3.4. Схеми за паралелно изпълнение	9
3.5. Управление на състоянието и на случайността	9
3.6. Точки, в които проектът очаква измерване	10
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	11
4.1. Организация на изходния текст и изграждане	11
4.2. Представяне на решението	11
4.3. Инкрементално оценяване	11
4.4. Паралелно изпълнение и синхронизация	11
4.5. Векторизация на оценяването	12
4.6. Тестване	12

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА	13
5.1. Изследвани въпроси	13
5.2. Апаратна и програмна среда	13
5.3. Еталонни инстанции	13
5.4. Критерий за спиране и изчислителен бюджет	14
5.5. Брой повторения и обобщаване	14
5.6. Измервани величини	14
5.7. Профилиране	15
5.8. Заплахи за валидността	15
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	16
6.1. Проверка на условията за валидност	16
6.2. Ускорение и ефективност спрямо броя нишки	16
6.3. Качество на решението при фиксиран бюджет	16
6.4. Сравнение между паралелните схеми	17
6.5. Разпределение на времето и профил	17
6.6. Обобщение на резултатите	17
ЗАКЛЮЧЕНИЕ	18
ЛИТЕРАТУРА	19
ПРИЛОЖЕНИЯ	20

УВОД

Голяма част от практически важните задачи за комбинаторна оптимизация са изчислително трудни, така че точното им решаване е приложимо само за малки входове. Вместо точно решение, инженерната практика използва *метаевристики*: общи стратегии за търсене, които не гарантират оптимум, но намират добро допустимо решение в приемливо време. Систематичният преглед в [1] показва, че тези стратегии се различават не по задачата, която решават, а по начина, по който балансират *изследване* на пространството от решения и *задълбочаване* в перспективен негов участък.

Точно тази обща форма прави метаевристичните подходящи за паралелно изпълнение: търсенето се състои от много слабо свързани опити, а не от една дълга зависима верига от изчисления. Класификацията на паралелните схеми в [2] обаче показва, че лесното паралелизиране е подвеждащо. Различните схеми, независимо многократен старт, споделяне на общо най-добро решение и островен модел с обмен между популации, се различават не само по постигнатото ускорение, а и по това дали изобщо решават същата задача: кооперативните схеми променят траекторията на търсенето, тоест променят и качеството на решението, а не само времето за получаването му.

Оттук следва и проблемът, около който е организиран настоящият курсов проект. Твърдение от вида „реализацията е паралелна и затова е по-бърза“ не е проверимо, докато не бъдат фиксирани едновременно три неща: наборът от еталонни входни инстанции, критерият за спиране и начинът, по който се сравнява качеството на получените решения. Без тези три условия измереното ускорение може да идва от по-слаба работа на нишка, а не от по-добро използване на машината.

Целта на проекта е да се проектира и реализира паралелна рамка за метаевристична оптимизация на комбинаторни задачи, в която няколко метаевристики и няколко схеми за паралелно изпълнение стоят зад общ договор, и да се направи количествена оценка на производителността и на качеството на решенията при контролирани условия.

За постигането на целта са формулирани следните **задачи**:

1. преглед на теорията на метаевристичните и на схемите за паралелното им изпълнение;
2. анализ на възможните проектни решения, тоест език и модел на паралелизъм, начин на представяне на задачата и стратегия за обмен на информация между нишките, и обосновка на направения избор;

3. проектиране на архитектура, в която задачата, метавристиката и схемата за паралелно изпълнение са три независими точки на разширение;
4. програмна реализация на избраните метавристики и схеми, с внимание към цената на синхронизацията и към разположението на данните в паметта;
5. изграждане на методика за измерване, която фиксира входните инстанции, критерия за спиране, броя повторения и начина на обобщаване на резултатите;
6. експериментално измерване на ускорението, на ефективността и на качеството на решенията, профилиране на реализацията и сравнителен анализ на алтернативите.

Обхватът на работата е рамката, метавристиките в нея и експерименталната оценка. Извън обхвата остават разпределеното изпълнение върху няколко машини, изпълнението върху графични ускорители и точните методи за решаване, които се използват само като източник на известни оптимални стойности за еталонните инстанции.

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането, а Раздел IV програмната реализация. Раздел V описва методиката за измерване, която е предпоставка за валидността на всичко след нея. Раздел VI представя получените резултати и анализа им, а заключението обобщава изводите и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Разделът е подреден от общото към частното: първо задачата, която метаевристиките решават, после отделните семейства метаевристики и техните първоизточници, и накрая схемите за паралелно изпълнение, които са същинският предмет на проекта.

1.1. Комбинаторна оптимизация и мястото на метаевристиките

Задачата за комбинаторна оптимизация се задава с крайно множество от допустими решения и с целева функция върху него. Множеството е крайно, но нараства толкова бързо с размера на входа, че изчерпателното му обхождане е неприложимо още при скромни размери. Точните методи заобикалят това чрез отсичане на цели подмножества, но при трудните задачи и те се изправят пред същия ръст.

Метаевристиките приемат друг компромис: отказват се от гаранцията за оптималност и в замяна получават контрол върху изразходваното време. Общото между тях е описано в [1] като стратегия за управление на подчинена евристика, която сама по себе си би заседнала в локален оптимум. Подробното систематично изложение на семействата, на техните компоненти и на въпросите по реализацията им е дадено в [3].

1.2. Метаевристики, включени в проекта

1.2.1. Симулирано закаляване

Симулираното закаляване приема и влошаващи ходове с вероятност, която намалява с понижаването на управляващ параметър, наречен температура по аналогия с металургичния процес. Методът е формулиран в [4], където е и връзката с алгоритъма на Метрополис. Практическото поведение на метода зависи изцяло от схемата на охлаждане и от дефиницията на съседството, което го прави удобна отправна точка за измервания: една и съща реализация дава различно качество при различна схема, и разликата е измерима.

1.2.2. Търсене с забрани

Търсенето с забрани, въведено в [5], използва краткосрочна памет за скорошните ходове и забранява връщането към тях за определен брой стъпки. Така търсенето се

измъква от локален оптимум, без да разчита на случайност. Забраненият списък е структура с честа проверка за принадлежност, тоест точката, в която представянето на данните в паметта пряко влияе върху времето за една итерация.

1.2.3. Генетичен алгоритъм

Популационните методи, чиято основа е положена в [6], поддържат множество решения едновременно и ги комбинират чрез оператори за кръстосване и мутация. За настоящия проект тези методи са важни с това, че оценката на пригодността на отделните индивиди е независима, тоест естествено се разпределя между нишки.

1.2.4. Мравчена колония

Оптимизацията с мравчена колония [7] изгражда решения стъпка по стъпка, като използва обща за колонията матрица от следи, обновявана след всяка итерация. Тази обща матрица е и основната разлика спрямо предходните три метода: тук споделеното състояние не е оптимизация, а част от самия алгоритъм, което прави метода естествен тест за цената на синхронизацията.

1.3. Паралелни схеми за изпълнение

Прегледът в [2] и сборникът [8] класифицират паралелните метаевристики по това какво точно се разпределя: отделни независими търсения, оценката на съседството в рамките на едно търсене, или популация, разделена на подпопулации с периодичен обмен. Трите варианта имат различни свойства. Независимият многократен старт не изисква комуникация и мащабира почти линейно, но не използва информацията, натрупана от останалите нишки. Кооперативните схеми използват тази информация, но плащат за нея със синхронизация и с риск от преждевременно сходимост към едно и също решение. Островният модел стои между двете и въвежда честотата на обмена като допълнителен управляващ параметър.

1.4. Оценка на производителността

Сравнението на паралелни реализации изисква модел, който отделя ограничението от изчисленията от ограничението от паметта. Моделът на покривната линия в [9] свързва постигнатата производителност с аритметичната интензивност на изчислението и дава

горна граница, спрямо която измерената стойност се тълкува. За метаевристичните този модел е приложим главно към оценяването на целевата функция, което е и най-често изпълняваната част от кода.

1.5. Еталонни набори от инстанции

Сравнимостта с публикувани резултати изисква публично достъпни инстанции с известни или най-добри известни стойности. За задачата на търговския пътник такъв набор е TSPLIB [10], а за редица други комбинаторни задачи, включително задачата за раницата и задачите за покриване на множество, такъв набор е OR-Library [11]. Конкретните инстанции, използвани в проекта, са изброени в Раздел V.

1.6. Изводи от прегледа

[TODO: Обобщение в 1 абзац: коя празнина в прегледаната литература адресира проектът. Формулира се след като бъдат прочетени изцяло източниците, а не преди това.]

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Проектът има четири независими проектни решения: език и среда за изпълнение, модел на паралелизъм, начин на представяне на задачата пред алгоритъма и стратегия за обмен на информация между нишките. Разделът разглежда алтернативите по всяко от тях и обосновава направения избор. Обосновката е предварителна там, където зависи от измерване, и в такива случаи е отбелязана изрично.

2.1. Език и среда за изпълнение

Разгледани са три възможности. Език с управлявана среда за изпълнение би съкратил времето за разработка, но въвежда събиране на боклука в най-горещия цикъл на търсенето и прави измерването на времето за една итерация зависимо от фактор извън контрола на програмата. Език с явно управление на паметта и без среда за изпълнение дава пълен контрол, но за част от нужните конструкции няма зряла стандартна поддръжка. Избран е C++ по стандарта ISO/IEC 14882:2020 [12], защото дава едновременно контрол върху разположението на данните в паметта и стандартни средства за паралелизъм, тоест позволява измерването да отразява алгоритъма, а не средата под него.

2.2. Модел на паралелизъм

Първата възможност е директно управление на нишки. Тя дава най-точен контрол върху свързването на нишка с ядро и върху момента на синхронизация, но прехвърля цялата координация върху програмиста. Втората е задачно ориентирана среда с крадене на работа, която се справя по-добре с неравномерно натоварване, но добавя слой, чиято цена трябва да бъде отчетена отделно при измерването. Третата е директивен подход над цикли, който е най-икономичен откъм код, но се вписва зле в търсене с нерегулярна структура.

Избран е първият вариант като основен, с явно управление на нишките и с атомарни операции за споделеното състояние, защото предметът на измерването е именно цената на синхронизацията и тя не бива да бъде скрита зад чужд планировчик. **[TODO: Да се реши дали задачно ориентираната среда влиза като втори, сравнителен вариант, или остава в предложенията за по-нататъшна работа.]**

2.3. Представяне на задачата пред алгоритъма

Възможните решения са две. Динамичен полиморфизъм чрез виртуални функции дава един общ договор и позволява алгоритъмът да бъде компилиран веднъж, но вкарва непряко извикване в най-често изпълнявания път, тоест в оценяването на целевата функция. Статичен полиморфизъм чрез шаблони и концепции премахва това извикване и отваря целевата функция за вграждане и за векторизация, но увеличава времето за компилация и размера на кода.

Избран е статичният вариант за горещия път и общ динамичен договор на нивото на конфигуриране на експеримента. Разделянето на двете нива е проектно решение, а не компромис: изборът на алгоритъм се прави веднъж на изпълнение, а оценяването на целевата функция се прави милиони пъти.

2.4. Стратегия за обмен между нишките

Трите разгледани схеми следват класификацията от [2]: независим многократен старт, кооперативна схема със споделено най-добро решение и островен модел с периодична миграция. Те не са взаимно изключващи се, а образуват редица с нарастваща комуникация, затова и трите са реализирани и влизат в сравнението. Твърдението, че по-голямата комуникация се изплаща с по-добро качество, е хипотеза за проверка, а не изходно допускане.

2.5. Обобщение на избора

[TODO: Таблица с колони: проектно решение, разгледани алтернативи, избрано решение, основание. Попълва се, след като изборът по §2.2 бъде окончателно потвърден.]

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Проектирането изхожда от едно изискване: трите неща, които се менят в експериментите, тоест задачата, метаевристиката и схемата за паралелно изпълнение, трябва да се менят независимо едно от друго. Ако смяната на схемата за изпълнение налага промяна в кода на алгоритъма, всяко сравнение между схемите става сравнение и между две различни реализации на алгоритъма, тоест престава да бъде измерване на това, което се твърди.

3.1. Слоеви и отговорности

Системата е разделена на четири слоя. Най-долният описва задачата и предоставя представяне на решение, целева функция и оператор за преминаване към съседно решение. Над него стои слойът на метаевристиките, който работи само през този договор и не знае коя конкретна задача решава. Третият слой съдържа схемите за паралелно изпълнение, които управляват няколко екземпляра на търсенето и обмена между тях. Най-горният слой е конфигурирането и измерването: избор на инстанция, на алгоритъм, на схема и на критерий за спиране, и записване на резултатите в машинно четим вид.

**[TODO: Фигура: диаграма на слоевете и на посоката на зависимостите (tikz).
Задължителна е препратка от текста с макроса \figref по A.9.]**

3.2. Договор за задача

Договорът за задача е минималният набор от операции, които всяка метаевристика изисква. Той включва тип на решението, оценяване на решението, генериране на начално решение и обхождане на съседството. Отделно се предвижда операция за *инкрементално оценяване*, тоест изчисляване на промяната в целевата функция при конкретен ход, без пълно преизчисляване. Тази операция не е удобство, а проектно решение с пряко следствие върху производителността: при задачата на търговския пътник пълното оценяване е с линейна сложност спрямо броя на върховете, а промяната при размяна на две ребра се изчислява с константен брой операции.

[TODO: Точният списък от операции в договора се фиксира след първата реализация на две задачи, за да не бъде обобщен от един частен случай.]

3.3. Договор за метабвристика

Метабвристиката се разглежда като обект със състояние, който изпълнява стъпки, а не като функция, която връща решение. Причината е в паралелните схеми: за да може една нишка да получи решение отбън и да продължи от него, търсенето трябва да бъде прекъсваемо в определени точки, а състоянието му да бъде видимо. Договорът включва изпълнение на една стъпка, четене на текущото най-добро решение, приемане на решение отбън и проверка за изпълнен критерий за спиране.

3.4. Схеми за паралелно изпълнение

И трите схеми използват един и същ договор за метабвристика и се различават само по това какво се обменя и кога.

- **Независим многократен старт.** Всяка нишка изпълнява самостоятелно търсене със собствен генератор на псевдослучайни числа. Обмен няма, а резултатът е най-доброто от всички решения. Тази схема е и еталонът, спрямо който се измерва цената на комуникацията в останалите две.
- **Кооперативна схема със споделено най-добро решение.** Нишките четат и обновяват обща най-добра стойност. Обновяването е рядко, а четенето е често, тоест схемата се проектира около това съотношение.
- **Островен модел.** Нишките са разделени на подпопулации, които обменят решения на определен брой итерации. Топологията на обмена и честотата му са параметри на експеримента.

3.5. Управление на състоянието и на случайността

Възпроизводимостта е изискване към проекта, а не пожелание. Всяка нишка получава собствен генератор на псевдослучайни числа с извлечено от общо начално число подсемя, така че едно изпълнение да може да бъде повторено точно при същия брой нишки. Изпълнение с различен брой нишки не е и не може да бъде побитово еднакво, затова сравненията между конфигурации се правят по разпределението на резултатите от няколко независими изпълнения, а не по единично изпълнение.

3.6. Точки, в които проектът очаква измерване

[TODO: Списък на точките за измерване: брой оценявания на целевата функция, време на итерация, време в изчакване по синхронизация, брой успешни и неуспешни атомарни обновявания. Списъкът се фиксира заедно с методиката в Раздел V.]

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Разделът описва как проектът от Раздел III е сведен до работеща програма и кои решения при реализацията имат пряко отражение върху измереното време.

4.1. Организация на изходния текст и изграждане

Изходният текст е разделен на заглавни файлове с договорите и с шаблонните части, които трябва да бъдат видими за компилатора в мястото на употреба, и на файлове с реализация за частите, които не са шаблонни. Изграждането е с CMake, като целите за измерване и за автоматизирани тестове са отделни от библиотеката, за да може библиотеката да се изгражда без зависимости за измерване.

[TODO: Дърво на директориите и кратко описание на всеки модул. Попълва се след като структурата се стабилизира, не преди това.]

4.2. Представяне на решението

Решението се пази в непрекъснат блок памет, а не във верижна структура. Причината е, че основната операция при търсенето е последователно обхождане при оценяването, тоест достъпът е предвидим и се възползва от предварителното зареждане в кеша. Съседството се задава като операция върху индекси, което позволява ходът да се приложи и да се отмени без копиране на цялото решение.

4.3. Инкрементално оценяване

Инкременталното оценяване е реализирано като отделна операция в договора за задача. За задачата на търговския пътник промяната при размяна на две ребра засяга краен брой разстояния и се изчислява без обхождане на целия маршрут. **[TODO: Листинг на инкременталното оценяване и кратък коментар кои са предпоставките за коректността му, тоест кога е допустимо да замести пълното оценяване.]**

4.4. Паралелно изпълнение и синхронизация

Нишките се управляват със стандартните средства на езика, а времето им на живот е обвързано с обхвата на изпълнявания обект, така че изход по изключение да не оставя

работещи нишки. Споделеното най-добро решение се обновява по схемата „сравни и размени“ в цикъл, а не под общо заключване, защото обновяването е рядко, а четенето е често. Отделните нишки работят върху свои данни, подравнени така, че две нишки да не пишат в една и съща кеш линия.

[TODO: Да се провери с измерване, че подравняването действително премахва фалшивото споделяне при тази задача, вместо да се предполага. Резултатът влиза в Раздел VI.]

4.5. Векторизация на оценяването

Оценяването на целевата функция е единственото място, където данните са достатъчно регулярни, за да позволят обработка на няколко елемента наведнъж. Реализацията разчита на автоматична векторизация при подходящо подредени данни, а ръчно написаните варианти се разглеждат само ако измерването покаже, че компилаторът не векторизира. **[TODO: Проверка на генерирания машинен код за горещия цикъл и заключение дали ръчен вариант е необходим.]**

4.6. Тестване

Тестовите покриват три групи твърдения: че инкременталното оценяване дава същия резултат като пълното за произволни ходове, че всяко върнато решение е допустимо за задачата, и че при фиксирано начално число и фиксиран брой нишки резултатът е възпроизводим. Тестовите за производителност са отделени от тестовите за коректност, защото първите са чувствителни към натоварването на машината, а вторите не бива да бъдат.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА

Методиката фиксира предварително какво се измерва, върху какво, при какви условия и как се обобщава. Целта е измерванията да са сравними помежду си и с публикувани резултати, и разликите да могат да бъдат отдадени на конкретна причина, а не на разлика в условията.

5.1. Изследвани въпроси

Експериментите отговарят на четири въпроса, всеки от които е формулиран така, че да може да бъде опроверган:

1. Как се променя времето до фиксиран критерий за спиране с нарастване на броя нишки и как се отнася полученото ускорение към идеалното?
2. Дава ли кооперативната схема по-добро качество на решението от независимия многократен старт при равен изчислителен бюджет?
3. Каква част от времето отива за оценяване на целевата функция и каква за синхронизация и изчакване?
4. Как се подреждат четирите метабевристики по качество на решението при еднакъв бюджет и върху едни и същи инстанции?

5.2. Апаратна и програмна среда

[TODO: Процесор, брой физически и логически ядра, размери на кешовете, размер и честота на паметта, операционна система и версия на ядрото, компилатор и версия, флагове за оптимизация. Записва се от реалната машина, не по документация.]

Измерванията се правят при спрян режим на динамично управление на честотата, ако средата позволява това, а машината не изпълнява друго натоварване. Всяко отклонение от тези условия се записва заедно с резултата, вместо да бъде премълчано.

5.3. Еталонни инстанции

Използват се публично достъпни инстанции с известни или най-добри известни стойности, за да бъде качеството на решението изразимо като относително отклонение,

а не като абсолютно число без мащаб. За задачата на търговския пътник източникът е TSPLIB [10], а за останалите разглеждани задачи източникът е OR-Library [11].

[TODO: Точен списък на избраните инстанции с имена, размери и известни оптимални или най-добри известни стойности, преписани от източника. Изборът покрива поне три порядъка по размер, за да се види къде мащабирането се променя.]

5.4. Критерий за спиране и изчислителен бюджет

Сравнението между конфигурации с различен брой нишки изисква еднаква мярка за изразходвана работа. Използват се два режима, като всеки резултат посочва в кой от двата е получен:

- **Фиксирано качество.** Измерва се времето до достигане на предварително зададено отклонение от известната стойност. Този режим отговаря на въпроса за ускорението.
- **Фиксиран бюджет.** Измерва се качеството, постигнато за фиксиран общ брой оценявания на целевата функция. Този режим отговаря на въпроса за качеството и е независим от бързината на машината.

[TODO: Стойности на прага за качество и на бюджета. Определят се след пробно изпълнение, така че задачата да не е нито тривиална, нито недостижима.]

5.5. Брой повторения и обобщаване

Метаевристиките са стохастични, тоест единично изпълнение не е резултат. Всяка конфигурация се изпълнява многократно с различни начални числа, а резултатът се представя с медиана и с междуквартилен размах, защото разпределението на времената не е симетрично и средната стойност се влияе силно от единични бавни изпълнения.

[TODO: Брой повторения на конфигурация. Определя се от наблюдаваното разсейване при пробно изпълнение, а не се избира предварително за удобство.]

5.6. Измервани величини

Записват се време на изпълнение, брой оценявания на целевата функция, стойност на най-доброто намерено решение и относително отклонение от известната стойност. От тях се изчисляват ускорението, тоест отношението на времето при една нишка към времето при няколко нишки, и ефективността, тоест ускорението, разделено на броя

нишки. Отделно се записват времето, прекарано в изчакване по синхронизация, и броят на неуспешните атомарни обновявания.

5.7. Профилиране

Профилирането се прави в два етапа. Първо се снима разпределението на времето по функции при представителна конфигурация, за да се установи кой участък от кода доминира. След това за доминиращия участък се снемат броячи за кешови пропуски и за инструкции на такт, а получената производителност се съпоставя с горната граница по модела на покривната линия [9], за да се определи дали изчислението е ограничено от паметта или от изчислителните единици.

[TODO: Име и версия на използвания профайлър и точните команди за снемане. Записват се дословно, за да е повторимо.]

5.8. Заплахи за валидността

Методиката признава три известни ограничения. Първо, резултатите са получени на една машина и не се пренасят автоматично върху друга архитектура. Второ, качеството на метаевристика зависи силно от настройката на параметрите ѝ, тоест сравнението между алгоритми е сравнение между конкретни настройки, а не между методите изобщо. Трето, при фиксиран изчислителен бюджет схемите с комуникация извършват по-малко оценявания за същото време, което трябва да бъде отчетено при тълкуването, а не приписано на самия алгоритъм.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя резултатите, получени по методиката от Раздел V. Всяка таблица и всяка фигура се придружава с коментар в текста и с препратка по A.8 и A.9. Резултат без тълкуване не се включва.

6.1. Проверка на условията за валидност

Преди същинските резултати се проверява, че условията от §5.2 са изпълнени: машината не изпълнява друго натоварване, разсейването между повторенията е в очакваните граници и резултатите при една нишка са възпроизводими при фиксирано начално число.

[TODO: Резултат от проверката. Ако някое условие не е изпълнено, това се записва тук, а не се премълчава.]

6.2. Ускорение и ефективност спрямо броя нишки

[TODO: Таблица: редове по брой нишки (1, 2, 4, 8, ... до броя логически ядра), колони по схема за паралелно изпълнение. Клетка: медианно време и междуквартилен размах.]

[TODO: Таблица: същата структура, но с ускорение и ефективност, изчислени спрямо изпълнението с една нишка.]

[TODO: Фигура: ускорение спрямо брой нишки, с нанесена линия на идеалното ускорение като отправна линия. Легендата е задължителна по A.9.]

[TODO: Коментар: къде кривата се отклонява от идеалната и коя измерена величина обяснява отклонението. Обяснение без подкрепящо измерване не се пише.]

6.3. Качество на решението при фиксиран бюджет

[TODO: Таблица: редове по инстанция, колони по метаевристика. Клетка: медианно относително отклонение от известната стойност.]

[TODO: Фигура: разпределение на постигнатото качество по повторения, за да се вижда разсейването, а не само медианата.]

[TODO: Коментар: подрежда ли се качеството еднакво при всички инстанции, или подредбата зависи от размера на инстанцията.]

6.4. Сравнение между паралелните схеми

[TODO: Таблица: за всяка схема, постигнато качество при равен изчислителен бюджет и измерено време, прекарано в синхронизация.]

[TODO: Коментар: изплаща ли се комуникацията. Отговорът е валиден само за измерените инстанции и се формулира с това ограничение.]

6.5. Разпределение на времето и профил

[TODO: Таблица: дял на времето по участъци, тоест оценяване на целевата функция, генериране на съседи, синхронизация, останало.]

[TODO: Съпоставка на измерената производителност на горещия участък с горната граница по модела на покривната линия и извод дали участъкът е ограничен от паметта или от изчисленията.]

6.6. Обобщение на резултатите

[TODO: Отговор на всеки от четирите въпроса от §5.1, по един абзац. Въпрос, на който измерванията не дават отговор, се отбелязва като неотговорен, а не се заобикаля.]

ЗАКЛЮЧЕНИЕ

В рамките на проекта е проектирана и реализирана паралелна рамка за метаевристична оптимизация на комбинаторни задачи, в която задачата, метаевристиката и схемата за паралелно изпълнение са три независими точки на разширение. Реализирани са четири метаевристики и три схеми за паралелно изпълнение, а към тях е изградена методика за измерване, която фиксира входните инстанции, критерия за спиране, броя повторения и начина на обобщаване.

[TODO: Обобщение на получените резултати в 1 до 2 абзаца, с числа от Раздел VI. Пише се последно, след като таблиците са попълнени.]

Предимства и недостатъци на решението

[TODO: Предимствата и недостатъците се формулират спрямо измереното, а не спрямо намерението. Недостатък, който измерванията показват, се записва тук дори когато е неудобен.]

Насоки за по-нататъшна работа

Работата може да бъде продължена в няколко посоки. Разпределеното изпълнение върху няколко машини поставя въпроса за обмена при съществено по-висока цена на комуникацията, тоест променя баланса, измерен тук. Автоматичната настройка на параметрите на метаевристиките би премахнала едно от ограниченията, признати в §5.8, а именно че сравнението е между конкретни настройки, а не между методите изобщо. Задачно ориентираната среда за изпълнение остава разгледана, но неизмерена алтернатива по §2.2.

ЛИТЕРАТУРА

- [1] C. Blum and A. Roli, “Metaheuristics in combinatorial optimization: Overview and conceptual comparison,” *ACM Computing Surveys*, vol. 35, no. 3, pp. 268–308, 2003.
- [2] T. G. Crainic and M. Toulouse, “Parallel meta-heuristics,” in *Handbook of Metaheuristics* (M. Gendreau and J.-Y. Potvin, eds.), pp. 497–541, Boston, MA: Springer, 2 ed., 2010.
- [3] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, NJ: John Wiley & Sons, 2009.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [5] F. Glover, “Future paths for integer programming and links to artificial intelligence,” *Computers & Operations Research*, vol. 13, no. 5, pp. 533–549, 1986.
- [6] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [7] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [8] E. Alba, ed., *Parallel Metaheuristics: A New Class of Algorithms*. Hoboken, NJ: John Wiley & Sons, 2005.
- [9] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [10] G. Reinelt, “TSPLIB — a traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [11] J. E. Beasley, “OR-Library: Distributing test problems by electronic mail,” *Journal of the Operational Research Society*, vol. 41, no. 11, pp. 1069–1072, 1990.
- [12] International Organization for Standardization, Geneva, *ISO/IEC 14882:2020 — Programming languages — C++*, 2020.

ПРИЛОЖЕНИЯ

Приложение А. Договори на рамката

[TODO: Изходен текст на договора за задача и на договора за метаетристика, вмъкнат с \lstinputlisting от include/anneal/.]

Приложение Б. Реализация на паралелните схеми

[TODO: Изходен текст на трите схеми за паралелно изпълнение.]

Приложение В. Използвани еталонни инстанции

[TODO: Таблица с имената на инстанциите, размера им, източника и известната или най-добрата известна стойност, преписани от TSPLIB и OR-Library.]

Приложение Г. Команди за възпроизвеждане на измерванията

[TODO: Точните команди за изграждане, за изпълнение на измерванията и за снемане на профил, заедно с версиите на инструментите. Целта е трето лице да може да повтори измерването без допълнителни обяснения.]

Приложение Д. Пълни таблици с измерените стойности

[TODO: Необобщените резултати от всички повторения. В Раздел VI стоят обобщенията, а тук стоят данните, от които те са получени.]